

FastWMR × G1 (IsaacLab) 코드 구조 설계 — 통합본

FastWMR.pdf 설계안 + DWL(kevinpark135/dwl) + Holosoma(amazon-far/holosoma) 반영 · 2026-07-15

이 문서는 세 차례에 걸친 설계 논의(① FastWMR 알고리즘 설계안, ② DWL 레포 반영, ③ Holosoma/FastSAC 원본 레포 반영)를 하나로 합친 최종 통합본이다. 이전 두 PDF의 내용을 전부 포함하면서, Holosoma가 공개한 FastSAC의 실제 하이퍼파라미터와 설계 선택을 반영해 구조와 설명을 갱신했다. 새로 추가된 "FastSAC Actor/Critic 업데이트 상세" 섹션(6절)에서 sac_update.py에 실제로 들어가야 할 손실 함수와 하이퍼파라미터를 구체적으로 정리했다.

1 세 참고 소스 요약

소스	이 구조에 기여한 내용
FastWMR.pdf (자체 설계안)	Hidden state staleness 문제, burn-in + truncated BPTT, 3-optimizer 분리(estimator/critic/actor), replay buffer 스키마, gradient cutoff
DWL (kevinpark135/dwl)	같은 G1+IsaacLab 실전 구현 레포. rewards.py/events.py 같은 파일 분리 관례, import 안전성 패턴, baseline task 병행 등록, 순수 헬퍼 모듈(gait.py) 패턴, README 기반 디버깅 로그 관습
Holosoma (amazon-far/holosoma) / FastSAC 논문	FastSAC 자체의 실제 하이퍼파라미터(γ , lr, batch size 등), config 카테고리 분리(algo/reward/observation/curriculum/randomization), penalty-curriculum의 정확한 동작 방식, symmetry augmentation, 미니멀 reward 철학

2 전체 디렉토리 트리 (최종)

■ 신규(holosoma 반영) ■ 위치 이동(DWL 반영) ■ 이름 변경

```

IsaacLab/
├── source/isaacsim_tasks/isaacsim_tasks/manager_based/locomotion/velocity/config/
│   ├── g1/                                     # 원본 보존
│   │   └── ...
│   └── fastwmr/
│       ├── __init__.py                        * g1 폴더 복제 + DWL/holosoma 패턴 반영
│       ├── rough_env_cfg.py                  # gym.register: FastWMR + FastSAC-Baseline 둘 다 등록
│       ├── flat_env_cfg.py                  # G1FastWMRRoughEnvCfg (G1RoughEnvCfg 상속)
│       ├── baseline_env_cfg.py              # estimator 단독 디버깅용
│       ├── actions.py                       # "FastSAC baseline" ablation 전용 task
│       ├── action_terms.py                  # (선택) 지연/모터 오프셋용 lightweight action cfg
│       ├── observations.py                  # (선택) 런타임 action term, USD import 격리
│       ├── rewards.py                       # policy(o_t)+privileged(s_t) 관측 전부 통합
│       ├── randomization.py                 # G1Rewards 재사용, holosoma식 <10함 미니멀 철학 반영
│       └── curriculum.py                    이름 변경(구 events.py) · friction/push/payload 샘플+적용+버퍼
│                                           신규 · holosoma의 penalty-curriculum 매커니즘 반영
│
│   ├── agents/
│   │   ├── __init__.py
│   │   └── fastwmr_cfg.py                   # 하이퍼파라미터 entry point
│
│   ├── fastwmr_algo/
│   │   ├── __init__.py                     (DWL의 dwl/rsl_rl/ 패턴대로 task 폴더 안에 위치)
│   │   ├── config.py                       holosoma 검증 하이퍼파라미터 기본값 반영
│   │   ├── buffers/
│   │   │   └── sequence_replay_buffer.py    # boundary-safe B_in+L+1 샘플링
│   │   ├── networks/
│   │   │   ├── history_encoder.py          # Eψ: forward_rollout()/forward_sequence() 분리
│   │   │   ├── decoder.py                  # Dψ: MSE+BCE+L1
│   │   │   ├── actor.py                    # πθ, joint-limit action bound, concat(o,s) 여부 명시
│   │   │   └── critic.py                   # Qφ1,Qφ2, avg-Q(no CDQ), LayerNorm, C51
│   │   ├── algo/
│   │   │   ├── rollout_worker.py           # MaskAndRandomize, h_roll/c_roll 관리
│   │   │   ├── estimator_update.py         # burn-in + unroll + L_recon
│   │   │   ├── sac_update.py               # * 6절 상세 - holosoma FastSAC 수식 확장
│   │   │   ├── symmetry_utils.py           신규(선택) · 좌우 대칭 augmentation
│   │   │   └── fastwmr_agent.py            # 3-optimizer 오케스트레이션
│   │   └── utils/
│   │       ├── env_wrapper.py              # obs/privileged 분리 adapter
│   │       ├── temporal_indexing.py        # episode_id/env_id/timestep 규약 단일화
│   │       └── logging_utils.py
│
│   ├── tests/
│   │   ├── test_randomization.py           이름 변경(구 test_events.py)
│   │   └── test_observations.py
│
│   └── README.md                           # Update Log (ablation/디버깅 결정 기록)
│
├── scripts/reinforcement_learning/fastwmr/
│   ├── train.py                            # --task Isaac-Velocity-Rough-G1-FastWMR-v0
│   ├── play.py                             # 체크포인트 평가 + perturbation ablation
│   └── cli_args.py
│
└── logs/fastwmr/
    ├── <run_name>/
    │   ├── checkpoints/
    │   └── config_snapshot.yaml

```

3 Task 설정 레이어 — 파일별 역할 (전체)

파일	역할
<code>__init__.py</code>	<code>gym.register()</code> 로 Isaac-Velocity-Rough-G1-FastWMR-v0 (및 -Play), Isaac-Velocity-Rough-G1-FastSAC-Baseline-v0 등록. FastWMR은 자체 off-policy 루프이므로 <code>rsl_rl/skrl</code> 같은 on-policy 프레임워크용 entry point는 등록하지 않는다.
<code>rough_env_cfg.py</code>	<code>G1FastWMRRoughEnvCfg(G1RoughEnvCfg)</code> . 원본이 꺼둔 <code>push_robot/add_base_mass/base_com</code> 이벤트를 재활성화하고, 관측 그룹에 <code>privileged</code> 그룹을 추가한다.
<code>flat_env_cfg.py</code>	지형 randomization 없는 평지 버전. estimator 단독 검증, burn-in 길이 ablation용.
<code>baseline_env_cfg.py</code>	estimator/replay 구조 없이 순수 FastSAC + 표준 proprioceptive observation만 쓰는 대조군 task. holosoma 논문이 공개한 검증된 하이퍼파라미터(6절 표 참고)를 그대로 적용해서, "FastWMR이 안 되면 estimator 통합 문제인지 아니면 하이퍼파라미터/구현 문제인지"를 가장 먼저 가를 수 있게 한다.
<code>actions.py</code> / <code>action_terms.py</code>	(선택) 시스템 지연/모터 오프셋을 partial observability 측에 포함하고 싶을 때 추가. 무거운 런타임 클래스를 분리해서 IsaacLab이 env cfg를 파싱하는 시점에 Isaac Sim/USD 모듈이 너무 일찍 로드되는 걸 막는다.
<code>observations.py</code>	policy 그룹(o_t) : 관측 위치/속도, base 각속도, 명령 속도, 이전 action. privileged 그룹(s_t) : base 선속도(ground truth), 발끝 마찰 계수, 가해진 push force/torque, payload 질량, 발-지면 contact — 이게 replay buffer의 reconstruction target이 된다. base 선속도와 height scan은 일부러 policy 그룹에서 빼고 privileged 그룹에만 둔다. privileged 항목은 전부 randomization.py 가 채운 <code>env.fastwmr_*</code> 버퍼를 읽기만 한

다.

rewards.py	기본은 config/g1 의 G1Rewards 를 재사용하되, holosoma 논문의 미니멀 reward 철학(선속도·각속도 추적, 발높이 추적, 기본자세 페널티, 발 평행/교차 방지, alive 보상, 상체 직립 페널티, action-rate 페널티 — 총 10항 미만)을 참고 템플릿으로 삼는다. 각 페널티 항의 weight는 직접 상수로 박아두지 않고 curriculum.py 가 계산한 스케일 값을 곱해서 사용한다.
randomization.py (구 events.py)	세 가지를 항상 같이 처리: ① 샘플링, ② 실제 물리 파라미터에 적용, ③ privileged 버퍼에 기록. <code>init_fastwmr_event_buffers</code> 가 <code>env.fastwmr_friction/push_force_torques/payload_mass</code> 를 만들고, 각 샘플링 함수가 매 스텝/인터벌마다 버퍼를 갱신한다. friction처럼 IsaacLab 표준 이벤트가 샘플값을 돌려주지 않는 경우, 실제 적용은 표준 이벤트에 맡기고 같은 분포에서 별도로 한 번 더 샘플링해서 기록 용 버퍼에 저장하는 우회가 필요할 수 있다(근사치).
curriculum.py (신규)	holosoma가 실제로 쓰는 penalty-curriculum 메커니즘을 그대로 채용: domain randomization 자체는 처음부터 최대 세기로 켜두고 , 대신 각 페널티성 리워드 항의 가중치 를 평균 episode 길이가 특정 threshold를 넘을 때마다 단계적으로 올린다. 이렇게 하면 학습 초반엔 "일단 쓰러지지 않고 속도 명령을 따라가는 것"에 집중하고, 어느 정도 버티기 시작하면 부드러운 동작을 요구하는 페널티가 강해지는 방식으로 자연스럽게 난이도가 올라간다. enabled 플래그와 level_up_threshold (평균 episode 길이 기준값)를 파라미터로 노출한다.
tests/test_randomization.py, test_observations.py	friction/push/payload 샘플→적용→버퍼 기록값 일치 검증, privileged 관측 그룹의 shape/순서가 s_t 스펙과 맞는지 검증.

중요: randomization 세기 커리큘럼이 아니라 페널티 가중치 커리큘럼

DWL 레포의 디버깅 로그를 보면 "초반에 push/friction을 세게 걸었더니 학습이 아예 안 됐다"는 기록이 있어서, 1차 반영 때는 randomization 세기 자체를 서서히 올리는 커리큘럼을 제안했었다. 그런데 holosoma 논문을 보면 실제로 검증된 방식은 그게 아니라 **randomization은 처음부터 최대 세기로 켜두고, 대신 페널티 리워드의 가중치를 episode 길이 기준으로 서서히 올리는 것이다**. 전자는 "환경을 봐준다", 후자는 "리워드 압박을 봐준다"는 차이가 있다. 둘 다 시도해볼 가치는 있지만, 논문으로 검증된 후자를 기본값으로 삼는 게 안전하다.

4 알고리즘 레이어 — 파일별 역할 (전체)

파일	역할
fastwmr_algo/ (전체 위치)	DWL이 <code>dwl/rsl_rl/</code> 처럼 커스텀 네트워크·알고리즘 코드를 task 폴더 바로 아래 두는 관례를 따른다. 단, <code>__init__.py</code>, <code>rough_env_cfg.py</code>, <code>observations.py</code>, <code>rewards.py</code>, <code>randomization.py</code>, <code>curriculum.py</code> 처럼 <code>env_cfg</code> 파싱 경로에 있는 파일은 가법게 유지하고, torch 네트워크/학습 루프는 <code>train.py</code> 에서만 import한다.
config.py	B_in, L, buffer capacity 같은 FastWMR 고유 파라미터에 더해, holosoma 논문이 공개한 FastSAC 검증 하이퍼파라미터를 기본값으로 채워둔다 (6절 표 참고). 이렇게 하면 estimator를 붙이기 전에 이미 한 번 검증된 값에서 시작하게 된다.
buffers/sequence_replay_buffer.py	저장: <code>o,a,r,o',s,s',terminated,truncated,episode_id,env_id,timestep,reset_boundary</code> . <code>boundary</code> 넘는 시퀀스는 배제. <code>hidden state/\$_t</code> 는 저장하지 않음(staleness 회피).
networks/history_encoder.py	<code>Eψ.forward_rollout()</code> (env별 h_roll/c_roll 갱신)과 <code>forward_sequence()</code> (학습 시 재forward)를 분리.
networks/decoder.py	<code>Dψ</code> . continuous(MSE)+discrete(BCE)+L1 정규화.
networks/actor.py	$\pi\theta$. joint-limit-aware action bound (관절 한계 – 기본 자세 각도) 적용. LayerNorm 포함. 입력은 <code>stop_grad(\$_t)</code> 이며, raw o_t를 함께 concat할지 여부는 6절에서 다룬다.
networks/critic.py	<code>Qφ1</code> , <code>Qφ2</code> + target critic. LayerNorm 포함, CDQ(min) 대신 평균(avg-Q) 사용, C51 distributional head.
algo/rollout_worker.py	<code>o_t=MaskAndRandomize(s_t) → h_roll,c_roll=Eψ(...) → \$_t=Dψ(h_roll) → a_t~πθ(stop_grad(\$_t)) → env.step → buffer 저장 → 끝난 env만 h_roll/c_roll 리셋 → estimator update 이후 recent cache로 재구축.</code>
algo/estimator_update.py	<code>burn-in(no_grad) → learning unroll → L_recon → estimator optimizer(1개) step.</code>
algo/sac_update.py	6절에서 상세 설명 . holosoma FastSAC의 critic/actor/entropy/target 업데이트 수식을 \$ 기반으로 확장한 내용이 여기 들어간다.
algo/symmetry_utils.py (신규, 선택)	holosoma가 쓰는 좌우 대칭 augmentation을 FastWMR에도 적용하고 싶을 때. 6절 마지막에서 estimator와의 상호작용 문제를 설명한다.
algo/fastwmr_agent.py	3개 optimizer(estimator/critic/actor)를 들고 있는 최상위 객체. update 순서: 시퀀스 샘플 → estimator_update → (\$ detach) → sac_update → entropy temp → target soft update.
utils/env_wrapper.py, temporal_indexing.py, logging_utils.py	obs/privileged 분리 adapter, episode/timestep 규약 단일화(DWL의 gait.py 패턴), loss/ablation 태그 기록.

5 실행 스크립트 레이어

파일	역할
train.py	<code>gym.make(...)</code> → <code>env_wrapper</code> → <code>networks/buffer/agent</code> 생성 → <code>rollout_worker.collect()</code> 와 <code>agent.update()</code> 반복.
play.py	체크포인트 평가. ablation 후보(perturbation on/off)를 CLI 인자로 받아 <code>-Play env_cfg</code> 를 덮어씀.

6 FastSAC Actor/Critic 업데이트 상세 (sac_update.py)

holosoma 논문(Seo, Sferrazza 등, "Learning Sim-to-Real Humanoid Locomotion in 15 Minutes")이 공개한 FastSAC의 실제 설계를 기준으로, FastWMR의 **sac_update.py**에 들어가야 할 내용을 정리했다. 핵심은 **vanilla FastSAC의 raw observation o 자리를 estimator가 만든 $\hat{s}$ (reconstructed world state)로 바꾸는 것**이고, 나머지 critic/actor 설계(평균 Q, LayerNorm, C51, joint-limit action bound 등)는 holosoma가 이미 검증한 그대로 유지한다.

6.1 유지해야 하는 holosoma의 핵심 설계 선택

설계 선택	내용
Critic 타겟	Clipped Double Q(min) 대신 두 target critic의 평균 을 사용 — LayerNorm과 함께 쓸 때 CDQ가 오히려 성능을 해친다는 것 이 논문의 관찰.
Actor 목적함수	Q1, Q2 둘 다의 평균 을 최대화 (min이나 Q1만이 아님) — critic의 평균-Q 철학을 actor loss에도 동일하게 적용.
정규화	Observation normalization + LayerNorm 둘 다 사용. 특히 LayerNorm이 고차원 humanoid task 안정화에 중요.
분포형 critic	C51 (quantile regression은 큰 배치에서 비용이 너무 커서 배제).
Action bound	joint-limit-aware: 각 관절의 (한계값 – 기본 자세각)을 그 관절의 action bound로 사용.
탐험(엔트로피)	pre-tanh 표준편차 상한 $\sigma_{\max}=1.0$ (일반적인 e^2 보다 작음), 온도 α 는 0.001의 낮은 값으로 초기화 후 auto-tuning.
목표 엔트로피	locomotion(단순 속도 추적)은 0.0, whole-body tracking처럼 어려운 태스크는 $- A /2$.
할인율 γ	단순 속도 추적 태스크는 $\gamma=0.97$, 어려운 whole-body tracking은 $\gamma=0.99$.
옵티마이저	Adam, lr=3e-4, weight decay=1e-3(0.1은 고차원 태스크엔 과하다는 관찰), $\beta_2=0.95$ (기본 0.99보다 낮춰서 큰 배치에서 안정화).
배치/업데이트 빈도	배치 크기는 최대 8K까지 크게 가져갈수록 유리, 시뮬레이션 1스텝당 gradient step 수(num_updates)도 늘릴수록 유리(단, 시뮬레이션 속도가 병목이 되는 지점까지).
대칭 augmentation	좌우 대칭 관측/행동을 미러링해서 데이터로 추가 활용 — 대칭 보행 수렴 속도 향상에 도움이 됨을 확인.

6.2 vanilla FastSAC 업데이트 (holosoma 원본)

매 환경 스텝마다 num_updates 회 아래를 반복한다 (o, a, o', r'만 있는 단순 buffer 기준):

```
타겟 계산 ( $\tilde{a}' \sim \pi\theta(\cdot|o')$ ):  
y = r' + ( $\gamma/2$ ) ·  $\sum_{i=1,2} [Q_{\phi_i}^{\text{target}}(o', \tilde{a}') - \alpha \cdot \log \pi\theta(\tilde{a}'|o')]$   
  
Critic 업데이트 (i = 1, 2 각각):  
 $\phi_i \leftarrow \phi_i - \nabla_{\phi_i} (1/|B|) \sum (Q_{\phi_i}(o, a) - y)^2$   
  
Actor 업데이트 ( $\tilde{a} \sim \pi\theta(\cdot|o)$ , reparameterization trick):  
 $\theta \leftarrow \theta + \nabla_{\theta} (1/(2|B|)) \sum \sum_{i=1,2} [Q_{\phi_i}(o, \tilde{a}) - \alpha \cdot \log \pi\theta(\tilde{a}|o)]$   
  
엔트로피 온도 업데이트:  
 $\alpha \leftarrow \alpha - \nabla_{\alpha} (1/|B|) \sum (H_{\text{target}} - H(o)) \cdot \alpha$   
  
Target critic soft update (i = 1, 2):  
 $\phi_i^{\text{target}} \leftarrow \rho \cdot \phi_i^{\text{target}} + (1-\rho) \cdot \phi_i$ 
```

6.3 FastWMR 확장 — sac_update.py에 실제로 들어갈 내용

estimator_update.py가 만들어 넘겨준 $\hat{s}_{\text{detached_seq}}$ (learning window 전체, 이미 stop_grad 처리됨)를 받아서, 위 수식의 o 자리를 전부 $\hat{s}$ 로 치환한다:

```
타겟 계산 ( $\tilde{a}' \sim \pi\theta(\cdot|\hat{s}')$ , i는 learning window 내 인덱스):  
y_i = r'_i + ( $\gamma/2$ ) ·  $\sum_{k=1,2} [Q_{\phi_k}^{\text{target}}(\hat{s}'_i, \tilde{a}'_i) - \alpha \cdot \log \pi\theta(\tilde{a}'_i|\hat{s}'_i)]$   
  
Critic 업데이트 (window 전체 평균):  
 $\phi_k \leftarrow \phi_k - \nabla_{\phi_k} (1/(|B| \cdot L)) \sum_i (Q_{\phi_k}(\hat{s}_i, a_i) - y_i)^2 \quad \text{for } k=1,2$   
  
Actor 업데이트:  
 $\theta \leftarrow \theta + \nabla_{\theta} (1/(2|B| \cdot L)) \sum_i \sum_{k=1,2} [Q_{\phi_k}(\hat{s}_i, \tilde{a}_i) - \alpha \cdot \log \pi\theta(\tilde{a}_i|\hat{s}_i)]$   
  
엔트로피 / target soft update: vanilla와 동일, 입력만  $\hat{s}$ 로 교체
```

estimator optimizer, critic optimizer, actor optimizer는 서로 완전히 분리되어 있으므로, 위 critic/actor 업데이트가 $\hat{s}$ 에 대해 미분되지 않도록(=estimator 쪽으로 그래디언트가 새지 않도록) `$\hat{s}$ _detached_seq`는 critic/actor에 들어가기 전에 반드시 `.detach()`된 텐서여야 한다. 이 detach 지점이 깨지면 critic/actor의 Bellman/policy 그래디언트가 estimator의 reconstruction loss와 섞여서 3-optimizer 분리 설계 전체가 무너진다.

6.4 열린 설계 결정: $\hat{s}$ 단독 vs concat(o, $\hat{s}$)

WMR/DreamWaQ 계열 아키텍처는 보통 actor/critic 입력으로 $\hat{s}$ 단독이 아니라 `[o_t,  $\hat{s}_t$ ]`를 concat해서 넣는다 — 추정치가 약간 부정확하더라도 현재 시점의 원본 proprioceptive 신호를 직접 skip-connection처럼 같이 주는 것이 안정성에 도움이 된다는 것이 일반적인 관찰이다. FastWMR.pdf 설계안 자체에는 이 부분이 명시돼 있지 않으므로, **actor.py/critic.py** 구현 전에 이 결정을 먼저 확정하는 게 좋다. 기본값으로는 `concat(o_t,  $\hat{s}_t$ )`를 추천한다.

6.5 Symmetry Augmentation과 estimator의 상호작용 (주의)

LSTM 기반 estimator에 대칭 augmentation을 그대로 적용하기 어려운 이유

holosoma의 대칭 augmentation은 관측/행동을 좌우로 뒤집어서 배치에 추가하는 방식인데, FastSAC처럼 순간 관측(o) 기반 정책에서는 간단히 적용된다. 하지만 FastWMR처럼 LSTM으로 시퀀스를 인코딩하는 구조에서는 **미러링된 시퀀스가 원본과 다른 hidden state 궤적을 만들어내므로**, 단순히 SAC loss 계산 직전에 o/a만 미러링해서 끼워 넣는 방식은 원칙적으로 맞지 않는다. 제대로 하려면 미러링된 raw observation 시퀀스를 LSTM에 별도로 통과시켜야 하고, 이는 학습 스텝당 estimator forward 연산량을 대략 두 배로 늘린다. 8주 일정을 고려하면, `symmetry_utils.py`는 처음엔 `use_symmetry=False`로 꺼두고 estimator 파이프라인이 안정화된 뒤에 켜보는 순서를 권장한다.

7 Holosoma에서 가져온 그 밖의 설계 교훈

1. config 카테고리를 명확히 분리한다

holosoma는 algo/reward/observation/curriculum/randomization/terrain/simulator/command/training/logger를 서로 다른 config 카테고리로 분리하고, CLI에서도 `--reward.terms...`, `--curriculum.setup-terms...`, `--randomization.setup-terms...`처럼 카테고리별로 오버라이드한다. FastWMR도 `randomization.py`와 `curriculum.py`를 분리한 것처럼, 이후 파라미터가 늘어나도 "이게 randomization 문제인지 curriculum 문제인지 reward 문제인지"를 파일 단위로 바로 좁힐 수 있게 이 분리를 유지하는 게 좋다.

2. 검증된 하이퍼파라미터에서 시작한다

FastSAC은 이미 G1에서 대규모로 검증된 γ/lr /weight decay/배치 크기 조합이 있다. FastWMR의 estimator 통합이 잘 안 될 때 "이게 estimator 문제인지 SAC 하이퍼파라미터 자체가 잘못된 건지"를 가르려면, 6.1절 표의 값에서 시작해서 estimator 관련 부분만 바뀌가는 순서가 안전하다.

3. 리워드는 최소한으로, 대신 커리큘럼으로 난이도를 조절한다

holosoma는 20개 이상의 리워드 항을 쓰는 기존 관행 대신 10개 미만의 최소한의 항만 쓰고, 대신 `penalty-curriculum`으로 학습 진행도에 맞춰 압박 강도를 조절한다. FastWMR도 리워드 항을 늘리기보다 **curriculum.py**의 스케줄을 조정하는 쪽으로 먼저 시도해보는 게 하이퍼파라미터 탐색 비용을 줄인다.

8 DWL에서 가져온 설계 교훈

1. Import 안전성: config 파싱 경로와 무거운 런타임 코드를 분리

IsaacLab이 태스크를 등록/파싱하는 시점에 Isaac Sim/USD 런타임 모듈이 너무 일찍 로드되면 SimulationApp 시작 전에 충돌할 수 있다. `fastwmr_algo/`의 torch 네트워크 코드는 **train.py**에서만 import한다.

2. Ablation은 flag보다 별도 task 등록이 더 안전하다

관측 스키마 자체가 달라지는 조건(FastSAC baseline 등)은 별도 gym task로, 알고리즘 내부 처리만 달라지는 조건(stored hidden state vs zero-init vs burn-in re-inference)은 `fastwmr_algo/config.py`의 flag로 구분한다.

3. 공유 규약은 순수 헬퍼 모듈 하나에만 정의

DWL의 `gait.py`처럼, FastWMR은 **temporal_indexing.py**에 episode/timestep 경계 규약을 한 곳에서만 정의해서 `rollout_worker/replay buffer/estimator_update`가 서로 다른 해석을 갖지 않게 한다.

4. README에 디버깅/ablation 결정을 계속 기록

`config/fastwmr/README.md`에 Update Log 섹션을 두고, curriculum 임계값이나 randomization 세기를 바꿀 때마다 "왜 바꿨는지"를 남긴다.

9 전체 데이터 흐름

- [0] **randomization.py**: reset/interval마다 `friction·push·payload` 샘플링 → 적용 → `env.fastwmr_*` 버퍼 기록 (세기는 항상 최대로 고정)
- [0'] **curriculum.py**: 평균 episode 길이를 관찰해서 **rewards.py**의 페널티 항 가중치를 단계적으로 상향
- [1] **train.py** 실행 → `config/fastwmr/__init__.py`에 등록된 task로 `gym.make()`

- [2] `env_wrapper.py`가 `observations.py`의 `policy(o_t)/privileged(s_t)` 그룹 분리
- [3] `rollout_worker.py`: `h_roll/c_roll` 갱신 → `ŝ_t` 복원 → action 생성 → `env.step` → `sequence_replay_buffer.py` 저장
- [4] `fastwmr_agent.py`가 `temporal_indexing.py` 규약대로 `B_in+L+1` 시퀀스 샘플
- [5] `estimator_update.py` → `sac_update.py` (6절 수식) 순서로 3-optimizer 업데이트
- [6] (선택) `symmetry_utils.py`가 켜져 있으면 미러링된 시퀀스를 별도로 forward해서 augmentation 배치 구성
- [7] `logging_utils.py`가 loss/ablation 태그 기록 → `play.py`가 체크포인트 평가, `baseline_env_cfg.py` task와 비교

10 다음 스텝 제안

- `baseline_env_cfg.py`를 6.1절 표의 holosoma 검증 하이퍼파라미터 그대로 먼저 돌려서, G1이 push/friction을 최대 세기로 켜 채로도 서 있을 수 있는지부터 확인한다.
- `curriculum.py`의 `level_up_threshold` 초기값은 holosoma 예시값(평균 episode length 기준)에서 시작해서, 8주 일정에 맞게 조정한다.
- estimator 파이프라인(burn-in, boundary-safe 샘플링, gradient cutoff)이 **tests/**를 통과하는 걸 먼저 확인한 다음에만 `use_symmetry=True` 를 시도한다 — 두 가지를 동시에 디버깅하면 원인 분리가 어려워진다.
- 6.4절의 `concat(o,ŝ)` 여부는 늦어도 `actor.py/critic.py` 구현 착수 전에 결정해서 인터페이스를 고정해야 한다.

FastWMR.pdf 설계안 + kevinpark135/dwl(IsaacLab G1 구현 레포) + amazon-far/holosoma(FastSAC 원본 레포 및 논문 "Learning Sim-to-Real Humanoid Locomotion in 15 Minutes", Seo · Sferrazza 외, Amazon FAR)를 참고해서 작성.

참고 링크: IsaacLab (<https://github.com/isaac-sim/IsaacLab>) · DWL (<https://github.com/kevinpark135/dwl>) · Holosoma (<https://github.com/amazon-far/holosoma>) · 논문 (<https://arxiv.org/abs/2512.01996>)